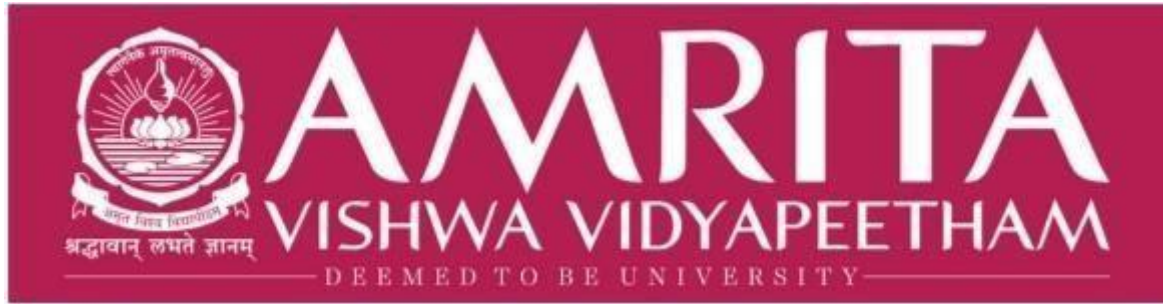




DATA STRUCTURE AND ALGORITHMS

23CSE203

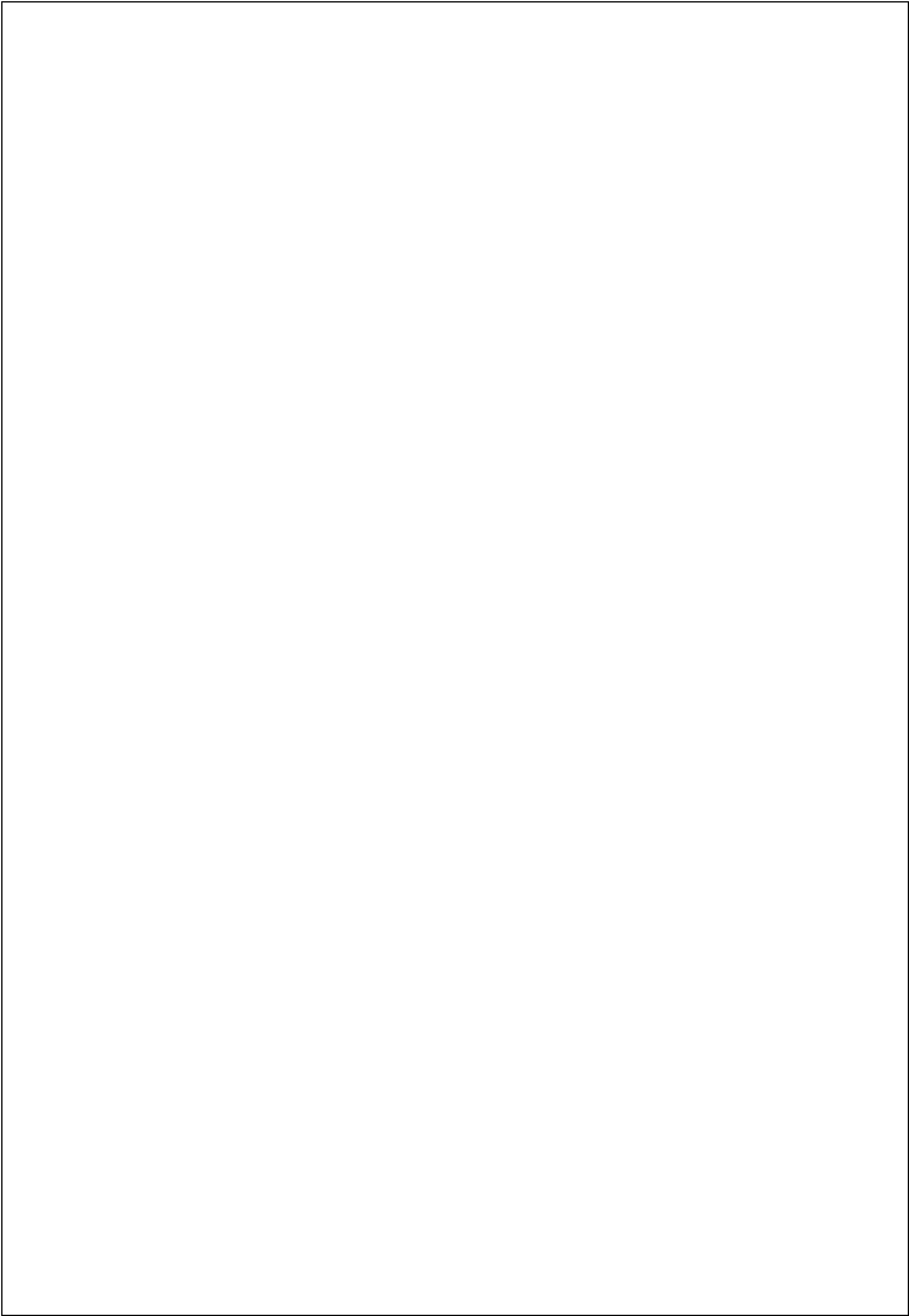
LAB MANUAL

SUBMITTED BY	
NAME	VARSHINI CHILLARA
ROLL NO	AV.SC.U4CSE25105
YEAR/SEM/SECTION	2nd YEAR/1/B
CREDITS	05

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

	TITLE	PG.NO	REMARKS
1	WEEK 1 : a) A rocket launch system displays a count down before launch. Instead of using a loop, implement the count down using recursion. Write a recursive python program to display countdown from n to 1 followed by launch. b) Banks calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n where P = Principal growth factor, n = number of years c) A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. d) In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula($n!$). Write a recursive formula to find the factorial of a given number e) Print first n terms in a fibanocci number series		
2	WEEK 2 : 1. Write a python program to implement linear search 2. Write a python program to implement binary search with And without sorted array array as an input 3. Write the implementation for the binary search algorithm along with the sorting the input elements if they are not sorted.		
3	WEEK 3 : a) Write a python program to implement bubble sort where the input elements are taken from the user. b) Write a python program to implement insertion sort where the input elements are taken from the user. c) Write a python program to implement selection sort where the input elements are taken from the user.		
4	WEEK 4 : a) Write a python program to implement merge sort where the input elements are taken from the user. b) Write a python program to implement quick sort where the input elements are taken from the user		
5	WEEK 5 : a) Write a Python program to implement a Singly Linked List (SLL) with the following operations: b) Write a Python program to implement a Double Linked List (DLL) with the following operations: c) Write a Python program to implement a Circular Linked List (CLL) with the following operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value		

	6. Delete Prior Node 7. Delete Last Node 8. Count Number of Nodes 9. Display/Traverse 10. Exit		
6	WEEK-6 : a) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array. b) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using Linked List.		
7	WEEK-7 : a) Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array and linked-list data structures. b) Circular Queue- its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array and linked-list data structures.		
8	WEEK-8 .		
10	WEEK-10 :		
11	WEEK 11 :		
12	WEEK 12 :		



LAB – 1

1.AIM : A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):  
    for i in range(n,-1,-1):  
        if i==0:  
            print("launch")  
        else:  
            print(i)  
n=int (input ("Enter  countdown number:"))  
launch(n)
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/countdowna.py =====  
Enter countdown number: 7  
7  
6  
5  
4  
3  
2  
1  
launch
```

EXPLANATION: This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

It uses a recursive function named countdown().

- The function calls itself with the value n-1.
- The recursion continues until n becomes 0.
- When n reaches 0, the program displays "Launch!".

ERROR TABLE:

S.No	Errors	Rectification
1.	Indentation Error	Use 3 spaces for each level of indentation
2.	Type Error	Convert the input to an integer using <code>n=int(input("Enter no,of seconds to launch:"))</code> .

1.B.AIM :

The banks calculate the growth of an investment using compound interest ;the formula requires calculating powers'.Which can be implemented recursively.Write a recursive program to calculate p^n (power),where p=principal growth factor, n = number of years.

PROGRAM:

```
def power (p, n):  
    if n == 0:  
        return 1  
    else:  
        return p * power (p, n - 1)  
p = float (input ("Enter principal/base value: "))  
n = int (input ("Enter number of years/powers: "))  
result = power (p, n)  
print("Power =", result)
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/interest.py =====  
Enter principal/base value: 2000  
Enter number of years/powers: 8  
Power = 2.56e+26  
,
```

EXPLANATION:

The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).

The recursion stops when n becomes 0, returning 1, which is the base case.

The final result is displayed after all recursive calls are completed.

- P represents the principal/growth amount.
- n represents the number of years.
- The user enters the values of P and n.
- The growth() function calls itself with n-1.
- When n becomes 0, the function returns 1.
- The result is calculated by repeatedly multiplying P.

ERROR TABLE:

S.No	Errors	Rectification
1	Type Error	Convert the input to float or int using float(input()) or (input()).
2.	Incorrect output If the recursive call is written as power(p, n+1) instead of power(p, n-1).	Change the recursive call to power(p,n-1) so that n decreases toward the base case.

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(lst, key, index):
    if index >= len(lst):
        return False

    if lst[index] == key:
        return True

    return search(lst, key, index + 1)

employees = [101, 102, 103, 104, 105]

key = int(input("Enter Employee ID to search: "))

if search(employees, key, 0):
    print(f"Employee ID found: {key}")
else:
    print("Employee ID Not Found")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/employee.py =====
Enter Employee ID to search: 102
Employee ID found: 102
```

EXPLANATION:

The recursive function checks each employee ID in the list one by one, starting from the first element.

If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.

If the entire list is searched without finding the ID, the function returns False.

ERROR TABLES:

S.no	Errors	Rectification
1.	Name Error if the list name is misspelled(e.g...employee instead of employees)'	Use the correct variable consistently throughout the program.
2.	Syntax Error due to a missing colon(:) after an if statement or function definition.	Add the missing colon(:) after if,else,or def.

4.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using factorial formula($n!$).develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a number: "))

if num < 0:
    print("Factorial does not exist for negative numbers.")
else:
    print("Factorial =", factorial(num))
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/factorial.py =====
Enter a number: 8
Factorial = 40320
|
```

EXPLANATION:

- The program calculates the factorial of a given number.
- Factorial is represented as $n!$.
- The factorial() function calls itself with $n-1$.
- If n is 1, the function returns 1.
- Otherwise, the current value of n is multiplied by the factorial of $n-1$.
- Negative or invalid values are handled by displaying a suitable message.

ERROR TABLE:

Sl.No	Errors	Rectification
1	Indentation Error if the statements inside the function are not properly indented	Maintain proper indentations(usually 4 spaces) inside the function
2	Type Error if a decimal (e.g,5.5) is entered for factorial calculation	Enter only a whole number or convert the input using int()

5.AIM

Write a recursive python program to print the first n terms of the Fibonacci series

PROGRAM:

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
```

```
n = int(input("Enter the number of terms: "))
```

```
print("Fibonacci Series:")
```

```
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/FIONAC
CI.PY
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
```

EXPLANATION:

- The program prints the first n terms of the Fibonacci series.
- The fib() function calculates Fibonacci numbers recursively.
- The first term is 0.
 - The second term is 1.
- Each subsequent term is calculated by adding the previous two terms.
 - The print_fib() function is used to print the Fibonacci terms.

ERROR TABLE:

S.no	Errors	Rectification
1.	Recursion Error if the base cases ($n == 0$ and $n == 1$) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB -2

1.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n = int(input("Enter no. of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):
    arr.append(int(input()))

key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found.")
```

OUTPUT:

```
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DS
A/linear search.py
Enter no. of elements: 7
Enter the elements:
23
46
67
78
98
96
36
Enter element to search: 98
Element found at index 4
>
```

EXPLANATION:

- The program implements the Linear Search algorithm.
- The user enters the number of elements.
- The elements are stored in an array/list.
- The user enters the element to be searched.
- The linear_search() function checks the elements one by one.
- If the element matches the key, its index is returned.
- If the element is not found, -1 is returned.

ERROR TABLE:

S.no	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input

PROGRAM:

```
def binary_search(arr, key):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == key:
```

```
            return mid
```

```
        elif arr[mid] < key:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
n = int(input("Enter no. of elements: "))
```

```
arr = []
```

```
print("Enter the elements:")
```

```
for i in range(n):
```

```
    arr.append(int(input()))
```

```
key = int(input("Enter element to search: "))
```

```
result = binary_search(arr, key)
```

```
if result != -1:
```

```
    print("Element found at index", result)
```

```
else:
```

```
    print("Element not found.")
```

OUTPUT:


```

===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/
Enter no. of elements: 6
Enter the elements:
23
36
39
90
36
34
Enter element to search: 90
Element not found.

```

EXPLANATION:

- The program implements Binary Search on a sorted array.
- The user enters elements in sorted order.
 - The algorithm uses low, high, and mid values.
- The middle element is calculated using $(low + high) // 2$.
 - If the middle element equals the search key, its index is returned.
 - If the key is greater than the middle element, the right half is searched.
 - If the key is smaller, the left half is searched.
 - If the element is not found, -1 is returned.

ERROR TABLE:

S.no	Errors	Rectification
1.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

2c)

Aim:

Write a program for implementation of binary search along with the sorting the elements

Program:

```
# Binary Search of Unsorted Array
# CSE25105

arr = []

# Read number of elements
n = int(input("Enter no of elements: "))

# Read elements
for i in range(n):
    element = int(input("Enter the element: "))
    arr.append(element)

# Check whether the list is sorted
if arr == sorted(arr):
    print("\nThe input list is already sorted.")
else:
    print("\nThe input list is not sorted.")
    print("\nSorting the list...")
    arr.sort()

# Display sorted list
print("\nThe sorted list is:", arr)

# Read element to search
key = int(input("\nEnter element to search: "))

# Binary Search Function
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
```

```

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1

# Perform binary search
result = binary_search(arr, key)

# Display result
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found.")

```

Output:

```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/SOTING BINARY SEARCH.py =
Enter no of elements: 7
Enter the element: 56
Enter the element: 67
Enter the element: 34
Enter the element: 89
Enter the element: 23
Enter the element: 45
Enter the element: 36

The input list is not sorted.

Sorting the list...

The sorted list is: [23, 34, 36, 45, 56, 67, 89]

Enter element to search: 67
Element found at index: 5
>>>

```

Error Table:

Error	Rectification
Used sorted method instead of which need to be referred to another variable as it returns a sorted array	Used the arr.sort() method which directly sorts the arr without using another variable

Description:

- The program performs Binary Search on user-provided elements.
- The input elements may initially be unsorted.
- The program checks whether the list is already sorted.
- If the list is not sorted, arr.sort() is used.
- The sorted list is displayed.
- The user enters the element to be searched.
- Binary Search is then performed on the sorted list.
- The middle element is compared with the search key.
- If the element is found, its index is displayed.
- Otherwise, "Element not found" is displayed.

LAB – 3

1.AIM : Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
```

```

        arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

# Taking input
n = int(input("Enter number of elements: "))

arr = []

print("Enter elements:")

for i in range(n):
    arr.append(int(input()))

bubble_sort(arr)

print("Sorted array:")

for element in arr:
    print(element, end=" ")

```

OUTPUT:

```

===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/BUBBLE SORT.py =====
Enter number of elements: 6
Enter elements:
34
56
90
87
56
23
Sorted array:
23 34 56 56 87 90

```

Explanation

Description:

The program implements the Bubble Sort algorithm.

- The user enters the number of elements.
- The elements are stored in a list.
- Adjacent elements are compared with each other.
- If the first element is greater than the second element, they are swapped.
- This process is repeated for multiple passes.
- Larger elements gradually move toward the end of the list.
- The bubble_sort() function performs the sorting.

Error table:

S.no	Error	Correction
1.	Used wrong loop range	Use for j in range(n-i-1):
2.	Swap condition is wrong	Use if arr[j] > arr[j+1]:

2.AIM:

Write a program to implement insertion sort where the input elements are taken from the user

PROGRAM:

```
# Insertion Sort
```

```
def insertion_sort(arr):  
    n = len(arr)
```

```
    for i in range(1, n):  
        key = arr[i]  
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j = j - 1
```

```
        arr[j + 1] = key
```

```
    return arr
```

```
# Taking input
```

```
n = int(input("Enter number of elements: "))
```

```
arr = []
```

```
print("Enter elements:")
```

```
for i in range(n):
```

```

arr.append(int(input()))

insertion_sort(arr)

print("Sorted array:")

for element in arr:
    print(element, end=" ")

```

OUTPUT:

```

===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/INSERTION.PY =====
Enter number of elements: 6
Enter elements:
34
56
08
20
05
89
Sorted array:
5 8 20 34 56 89
|

```

Explanation:

- The program implements the Insertion Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The first element is considered as the initially sorted portion.
 - Each subsequent element is stored in a variable called key.
- The key is compared with the elements before it.
- Elements greater than the key are shifted one position to the right.
 - The key is inserted into its correct position.
 - This process continues until all elements are sorted.

Error table:

S.no	Error	Correction
1.	key is not stored properly	Use key = arr[i]
2.	arr[j+1] = key placed inside while loop	Place it after the while loop

3.AIM :

Write a python program to implement selection sort where the input elements are taken from the user.

PROGRAM:

Selection Sort

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

# Taking input
n = int(input("Enter number of elements: "))

arr = []

print("Enter elements:")

for i in range(n):
    arr.append(int(input()))

selection_sort(arr)

print("Sorted array:")

for element in arr:
    print(element, end=" ")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/SELECTION SORT.py
Enter number of elements: 7
Enter elements:
23
89
67
47
36
90
45
Sorted array:
23 36 45 47 67 89 90
|
```

Explanation:

- The program implements the Selection Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The algorithm starts from the first unsorted element.
- The smallest element in the remaining list is identified.
- Its index is stored in min_index.
- The smallest element is swapped with the current element.
- The process is repeated for all positions in the list.

Error table:

S.no	Error	Correction
1.	min_index is not initialized	Use min_index = i
2.	Swap is placed inside inner loop	Place swap after the inner loop

LAB-4

4a)

Aim:

Write a python program to implement merge sort where the input elements are taken from the user.

Program

```
def merge_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = arr[:mid]
```

```
    right = arr[mid:]
```

```
    merge_sort(left)
```

```
    merge_sort(right)
```

```
    i = j = k = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] <= right[j]:
```

```
            arr[k] = left[i]
```

```
            i += 1
```

```
        else:
```

```
            arr[k] = right[j]
```

```
            j += 1
```

```
        k += 1
```

```
    while i < len(left):
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

```
    k += 1
```

```
n = int(input("Enter number of elements: "))
```

```
arr = []
```

```
print("Enter elements:")
```

```
for i in range(n):
```

```
    arr.append(int(input()))
```

```
merge_sort(arr)
```

```
print("Sorted Array:")
```

```
for element in arr:
```

```
    print(element, end=" ")
```

Output:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/merge_sort.py =====
Enter number of elements: 5
Enter elements:
45
56
67
90
23
Sorted Array:
23 45 56 67 90
```

Error table

error	rectification
Type Error : object of type 'NoneType' has no len()	Taken return arr at end of merge_sort

Explanation:

- The program implements the Merge Sort algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The `merge_sort()` function recursively divides the array into two halves.
- The array is divided until each sub-array contains one or zero elements.
- The left and right portions are then sorted recursively.
- The sorted left and right arrays are compared element by element.
- The smaller element is placed into the original array.
- Remaining elements from either portion are then copied into the array.
- Finally, the completely sorted array is displayed.
- Merge Sort follows the divide-and-conquer approach.

4b)

Aim:

Write a python program to implement quick sort where the input elements are taken from the user.

```
def quicksort(a, low, high):
```

```
    if low < high:
```

```
        i = low + 1
```

```
        j = high
```

```
        pivot = low
```

```
    while i <= j:
```

```
        while i <= high and a[i] <= a[pivot]:
```

```
            i += 1
```

```
        while a[j] > a[pivot]:
```

```
            j -= 1
```

```
    if i < j:
```

```
        a[i], a[j] = a[j], a[i]
```

```
    a[j], a[pivot] = a[pivot], a[j]
```

```
    quicksort(a, low, j - 1)
```

```
    quicksort(a, j + 1, high)
```

```
a = list(map(int, input("Enter numbers to sort: ").split()))
```

```
n = len(a)
```

```
quicksort(a, 0, n - 1)
```

```
print("Sorted array:")
```

OUTPUT:

```
> ===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/quicksort.py =====
Enter numbers to sort: 23 56 12 78 45
Sorted array:
[12, 23, 45, 56, 78]
>
```

Error table

Error	Rectification
$i < \text{len}(a)$ checks the complete array instead of current subarray	Use $i \leq \text{high}$ and maintain correct partition boundaries

Description:

- The program implements the Quick Sort algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The `quick_sort()` function is used to sort the array recursively.
- The first element of the current portion is selected as the pivot.
- Two pointers, i and j , are used to compare elements with the pivot.
- i moves from the left side and searches for an element greater than or equal to the pivot.
- j moves from the right side and searches for an element smaller than or equal to the pivot.
- If $i < j$, the two elements are swapped.
- After the partitioning process, the pivot is placed in its correct position.
- The array is then divided into left and right portions.
- Quick Sort is recursively applied to both portions.
- Finally, the sorted array is printed.

Week-5

5a)

Aim: ➤ Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Node
9. Display/Traverse
10. Exit Program:

Program:

Singly Linked List

CSE25105

print("Implementation of SLL")

Node Class

class Node:

def __init__(self, data):

self.data = data

self.next = None

Singly Linked List Class

class SingleLinkedList:

def __init__(self):


```
self.head = None
```

```
# Creation
```

```
def create(self):
```

```
    n = int(input("Enter no of elements: "))
```

```
    for i in range(n):
```

```
        data = int(input("Enter the value: "))
```

```
        new = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next:
```

```
                temp = temp.next
```

```
            temp.next = new
```

```
# Insert at Beginning
```

```
def insert_begin(self, data):
```

```
    new = Node(data)
```

```
    new.next = self.head
```

```
    self.head = new
```

```
# Insert at End
```

```
def insert_end(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

else:

temp = self.head

while temp.next:

temp = temp.next

temp.next = new

Insert at Specific Index

def insert_index(self, index, data):

if index == 0:

self.insert_begin(data)

return

elif index < 0:

print("Invalid index")

return

else:

new = Node(data)

temp = self.head

for i in range(index - 1):

if temp is None or temp.next is None:

print("Invalid index")

return

temp = temp.next

new.next = temp.next

temp.next = new

```
# Delete Specific Value
```

```
def delete(self, data):
```

```
    if self.head is None:
```

```
        print("No data")
```

```
        return
```

```
    temp = self.head
```

```
    # If first node contains the value
```

```
    if temp.data == data:
```

```
        self.head = temp.next
```

```
        print("Value deleted")
```

```
        return
```

```
    # Search for the value
```

```
    while temp.next and temp.next.data != data:
```

```
        temp = temp.next
```

```
    if temp.next is None:
```

```
        print("Value not present")
```

```
    else:
```

```
        temp.next = temp.next.next
```

```
        print("Value deleted")
```

```
# Delete Beginning
```

```
def delete_begin(self):
```

```
    if self.head is None:
```

```
        print("No data to delete")
```

```
    else:
```

```
temp = self.head
self.head = temp.next
print("Deleted value =", temp.data)
```

Delete End

```
def delete_end(self):
```

```
    if self.head is None:
        print("No data to delete")
```

```
    elif self.head.next is None:
        print("Deleted value =", self.head.data)
        self.head = None
```

```
    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted value =", temp.next.data)
        temp.next = None
```

Count Nodes

```
def count(self):
```

```
    if self.head is None:
        print("No data to count")
```

```
    else:
        count = 0
        temp = self.head
```

```
while temp:

    count += 1

    temp = temp.next

print(f"Number of nodes = {count}")
```

Display / Traverse

```
def display(self):
```

```
    if self.head is None:
```

```
        print("No data")
```

```
    else:
```

```
        temp = self.head
```

```
        while temp:
```

```
            print(temp.data, end=" --> ")
```

```
            temp = temp.next
```

```
        print("None")
```

Main Program

```
SLL = SingleLinkedList()
```

```
while True:
```

```
    print("\n----- SINGLY LINKED LIST -----")
```

```
    print("1. Create SLL")
```

```
    print("2. Insert at beginning")
```

```
    print("3. Insert at ending")
```

```
    print("4. Insert at index")
```

```
    print("5. Delete by value")
```

```
print("6. Delete first node")
print("7. Delete last node")
print("8. Count no. of nodes")
print("9. Display / Traverse")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    SLL.create()
    print("SLL is created")

elif choice == 2:
    data = int(input("Enter data to insert: "))
    SLL.insert_begin(data)
    print("Data is inserted at beginning")

elif choice == 3:
    data = int(input("Enter data to insert: "))
    SLL.insert_end(data)
    print("Data is inserted at ending")

elif choice == 4:
    data = int(input("Enter data to insert: "))
    index = int(input("Enter the index: "))
    SLL.insert_index(index, data)

elif choice == 5:
    data = int(input("Enter data to delete: "))
    SLL.delete(data)

elif choice == 6:
```

```
SLL.delete_begin()

elif choice == 7:
    SLL.delete_end()

elif choice == 8:
    SLL.count()

elif choice == 9:
    SLL.display()

elif choice == 10:
    print("Exiting the program...")
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:0)
Enter "help" below or click "Help" above for more informa
>>>
= RESTART: C:/Users/Varshini Chillara/AppData/Local/Progr
Implementation of SLL

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter no of elements: 3
Enter the value: 1
Enter the value: 3
Enter the value: 4
SLL is created

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data to insert: 5
Data is inserted at beginning

----- SINGLY LINKED LIST -----
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
```



```
Enter your choice: 2
Enter data to insert: 5
Data is inserted at beginning
```

```
----- SINGLY LINKED LIST -----
```

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

```
Enter your choice: 3
Enter data to insert: 7
Data is inserted at ending
```

```
----- SINGLY LINKED LIST -----
```

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

```
Enter your choice: 4
Enter data to insert: 4
Enter the index: 3
```

```
----- SINGLY LINKED LIST -----
```

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

```
Enter your choice: 5
Enter data to delete: 5
```

----- SINGLY LINKED LIST -----

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

Enter your choice: 6

Deleted value = 1

----- SINGLY LINKED LIST -----

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

Enter your choice: 7

Deleted value = 7

----- SINGLY LINKED LIST -----

1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit

Enter your choice: 8

Number of nodes = 3

```
----- SINGLY LINKED LIST -----
```

```
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
3 --> 4 --> 4 --> None
```

```
----- SINGLY LINKED LIST -----
```

```
1. Create SLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Exiting the program...
```

Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

- The program implements a **Singly Linked List** using Python classes.
- The **Node** class is used to create individual nodes.
- Each node contains two parts: **data** and **next**.
- **data** stores the value of the node.
- **next** stores the reference to the next node.
- The **SingleLinkedList** class manages the linked list.
- The **head** variable stores the reference to the first node.
- The program provides an option to **create** a linked list.
- It supports **insertion at the beginning** of the list.
- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** all the nodes.
- A menu-driven approach is used to allow the user to select different operations.
- The program continues running until the user selects the **Exit** option.

5b)

Aim:

➤ Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
# Doubly Linked List
```

```
# CSE25106
```

```
print("Implementation of DLL")
```

```
# Node Class
```

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.prev = None
```

```
self.next = None
```

```
# Doubly Linked List Class
```

```
class DoubleLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
# Creation
```

```
def create(self):
```

```
    n = int(input("Enter no of elements: "))
```

```
    for i in range(n):
```

```
        data = int(input("Enter the value: "))
```

```
        new = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next:
```

```
                temp = temp.next
```

```
            temp.next = new
```

```
            new.prev = temp
```

Insert at Beginning

```
def insert_begin(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
    else:
```

```
        new.next = self.head
```

```
        self.head.prev = new
```

```
        self.head = new
```

Insert at End

```
def insert_end(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next:
```

```
            temp = temp.next
```

```
        temp.next = new
```

```
        new.prev = temp
```

Insert at Specific Index

```
def insert_index(self, index, data):
```

```
    if index < 0:
```

```
        print("Invalid index")
```

```
        return
```

```
    if index == 0:
```

```
        self.insert_begin(data)
```

```
        return
```

```
    if self.head is None:
```

```
        print("Invalid index")
```

```
        return
```

```
    temp = self.head
```

```
    for i in range(index - 1):
```

```
        if temp.next is None:
```

```
            print("Invalid index")
```

```
            return
```

```
        temp = temp.next
```

```
    new = Node(data)
```

```
new.next = temp.next
```

```
new.prev = temp
```

```
if temp.next is not None:
```

```
    temp.next.prev = new
```

```
temp.next = new
```

```
# Delete Specific Value
```

```
def delete(self, data):
```

```
    if self.head is None:
```

```
        print("No data")
```

```
        return
```

```
    temp = self.head
```

```
# Search for the value
```

```
while temp and temp.data != data:
```

```
    temp = temp.next
```

```
if temp is None:
```

```
    print("Value not present")
```

```
    return
```



```
# If deleting the first node
if temp == self.head:
    self.head = temp.next

    if self.head is not None:
        self.head.prev = None

else:
    # Connect previous node to next node
    temp.prev.next = temp.next

    # Connect next node to previous node
    if temp.next is not None:
        temp.next.prev = temp.prev

print("Value deleted")
```

```
# Delete Beginning
```

```
def delete_begin(self):
```

```
    if self.head is None:
        print("No data to delete")
        return
```

```
    temp = self.head
```

```
    self.head = temp.next
```

```
if self.head is not None:
```

```
    self.head.prev = None
```

```
print("Deleted value =", temp.data)
```

```
# Delete End
```

```
def delete_end(self):
```

```
    if self.head is None:
```

```
        print("No data to delete")
```

```
        return
```

```
    temp = self.head
```

```
# Only one node
```

```
if temp.next is None:
```

```
    print("Deleted value =", temp.data)
```

```
    self.head = None
```

```
    return
```

```
# Move to last node
```

```
while temp.next:
```

```
    temp = temp.next
```

```
print("Deleted value =", temp.data)
```

```
temp.prev.next = None
```

```
# Count Nodes
```

```
def count(self):
```

```
    if self.head is None:
```

```
        print("No data to count")
```

```
        return
```

```
    count = 0
```

```
    temp = self.head
```

```
    while temp:
```

```
        count += 1
```

```
        temp = temp.next
```

```
    print("Number of nodes =", count)
```

```
# Display / Traverse
```

```
def display(self):
```

```
    if self.head is None:
```

```
        print("No data")
```

```
        return
```

```
temp = self.head
```

```
while temp:
```

```
    print(temp.data, end=" <--> ")
```

```
    temp = temp.next
```

```
print("None")
```

```
# Main Program
```

```
DLL = DoubleLinkedList()
```

```
while True:
```

```
    print("\n----- DOUBLY LINKED LIST -----")
```

```
    print("1. Create DLL")
```

```
    print("2. Insert at beginning")
```

```
    print("3. Insert at ending")
```

```
    print("4. Insert at index")
```

```
    print("5. Delete by value")
```

```
    print("6. Delete first node")
```

```
    print("7. Delete last node")
```

```
    print("8. Count no. of nodes")
```

```
    print("9. Display / Traverse")
```

```
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
```

```
    DLL.create()
```

```
    print("DLL is created")
```

```
elif choice == 2:
```

```
    data = int(input("Enter data to insert: "))
```

```
    DLL.insert_begin(data)
```

```
    print("Data is inserted at beginning")
```

```
elif choice == 3:
```

```
    data = int(input("Enter data to insert: "))
```

```
    DLL.insert_end(data)
```

```
    print("Data is inserted at ending")
```

```
elif choice == 4:
```

```
    data = int(input("Enter data to insert: "))
```

```
    index = int(input("Enter the index: "))
```

```
    DLL.insert_index(index, data)
```

```
elif choice == 5:
```

```
    data = int(input("Enter data to delete: "))
```

```
    DLL.delete(data)
```

```
elif choice == 6:
```

```
DLL.delete_begin()
```

```
elif choice == 7:
```

```
    DLL.delete_end()
```

```
elif choice == 8:
```

```
    DLL.count()
```

```
elif choice == 9:
```

```
    DLL.display()
```

```
elif choice == 10:
```

```
    print("Exiting the program")
```

```
    break
```

```
else:
```

```
    print("Invalid choice")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/double linked list.py =====
Implementation of DLL

----- DOUBLY LINKED LIST -----
1. Create DLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter no of elements: 4
Enter the value: 3
Enter the value: 5
Enter the value: 8
Enter the value: 9
DLL is created

----- DOUBLY LINKED LIST -----
1. Create DLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
```

Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

- The program implements a Doubly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains three parts: data, prev, and next.
- data stores the value of the node.
- prev stores the reference to the previous node.
- next stores the reference to the next node.
- The DoubleLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a doubly linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes. 36
- It provides a function to display/traverse the list.
- During insertion and deletion, both prev and next links are updated.
- The list is displayed using <--> to show the connection between nodes.
- A menu-driven approach is used to perform different DLL operations.
- The program continues until the user selects the Exit option.

5c)

Aim:

➤ Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
# Circular Linked List
# CSE25105
```

```
print("Implementation of CLL")
```

```
# Node Class
```

```
class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
# Circular Linked List Class
```

```
class CircularLinkedList:
```

```
    def __init__(self):
        self.head = None
```

```
# Creation
```

```
def create(self):
```

```
    n = int(input("Enter no of elements: "))
```

```
    for i in range(n):
```

```
        data = int(input("Enter the value: "))
```

```
        new = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new
```

```
            new.next = self.head
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next != self.head:
```

```
                temp = temp.next
```

```
            temp.next = new
```

```
            new.next = self.head
```

```
# Insert at Beginning
```

```
def insert_begin(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
        new.next = self.head
```

```
    else:
```



```

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new.next = self.head
        self.head = new
        temp.next = self.head

# Insert at End
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new
        new.next = self.head

# Insert at Specific Index
def insert_index(self, index, data):

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid index")
        return

    temp = self.head

    for i in range(index - 1):
        if temp.next == self.head:
            print("Invalid index")
            return

        temp = temp.next

    new = Node(data)
    new.next = temp.next
    temp.next = new

# Delete Specific Value
def delete(self, data):

    if self.head is None:
        print("No data")
        return

    # If head contains the value
    if self.head.data == data:

        # Only one node
        if self.head.next == self.head:

```

```

        self.head = None
        print("Value deleted")
        return

    # More than one node
    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    temp.next = self.head.next
    self.head = self.head.next

    print("Value deleted")
    return

    # Search for the value
    temp = self.head

    while temp.next != self.head and temp.next.data != data:
        temp = temp.next

    if temp.next == self.head:
        print("Value not present")
    else:
        temp.next = temp.next.next
        print("Value deleted")

# Delete Beginning
def delete_begin(self):

    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deleted value =", self.head.data)
        self.head = None
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    print("Deleted value =", self.head.data)

    temp.next = self.head.next
    self.head = self.head.next

# Delete End
def delete_end(self):

    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deleted value =", self.head.data)
        self.head = None
        return

    temp = self.head

```

```

while temp.next.next != self.head:
    temp = temp.next

print("Deleted value =", temp.next.data)

temp.next = self.head

# Count Nodes
def count(self):

    if self.head is None:
        print("No data to count")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes =", count)

# Display / Traverse
def display(self):

    if self.head is None:
        print("No data")
        return

    temp = self.head

    while True:
        print(temp.data, end=" --> ")
        temp = temp.next

        if temp == self.head:
            break

    print("HEAD")

# Main Program
CLL = CircularLinkedList()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create CLL")
    print("2. Insert at beginning")
    print("3. Insert at ending")
    print("4. Insert at index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

```

```
if choice == 1:
    CLL.create()
    print("CLL is created")

elif choice == 2:
    data = int(input("Enter data to insert: "))
    CLL.insert_begin(data)
    print("Data is inserted at beginning")

elif choice == 3:
    data = int(input("Enter data to insert: "))
    CLL.insert_end(data)
    print("Data is inserted at ending")

elif choice == 4:
    data = int(input("Enter data to insert: "))
    index = int(input("Enter the index: "))
    CLL.insert_index(index, data)

elif choice == 5:
    data = int(input("Enter data to delete: "))
    CLL.delete(data)

elif choice == 6:
    CLL.delete_begin()

elif choice == 7:
    CLL.delete_end()

elif choice == 8:
    CLL.count()

elif choice == 9:
    CLL.display()

elif choice == 10:
    print("Exiting the program")
    break

else:
    print("Invalid choice")
```

Output:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/c:
Implementation of CLL

----- CIRCULAR LINKED LIST -----
1. Create CLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter no of elements: 4
Enter the value: 2
Enter the value: 4
Enter the value: 6
Enter the value: 8
CLL is created

----- CIRCULAR LINKED LIST -----
1. Create CLL
2. Insert at beginning
3. Insert at ending
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: |
```

Error Table:

Error	Rectification
No major error is present in the given program.	No major rectification is required.

Description:

- The program implements a **Circular Linked List** using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The CircularLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to **create** a circular linked list.
- It supports **insertion at the beginning** of the list.
- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** the circular linked list.
- During traversal, the program stops when it reaches the head again.
- The list is displayed using --> and ends with **HEAD**, showing its circular nature.
- The program handles the special case where the list contains only one node.
- A **menu-driven approach** is used to perform different CLL operations.
- The program continues until the user selects the **Exit** option.

Week-6

6a)

Aim:

➤ Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Program:

```
# Stack using Array
# CSE25105
```

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    # Push operation
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    # Pop operation
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    # Peek operation
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack:")

            for i in range(self.top, -1, -1):
                print(self.stack[i])

# Main Program

size = int(input("Enter the size of the stack: "))

s = Stack(size)

while True:
```

```
print("\n----- STACK MENU -----")
print("1. Push")
print("2. Pop")
print("3. Peek")
print("4. Display")
print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    item = int(input("Enter the element to push: "))
    s.push(item)

elif choice == 2:
    s.pop()

elif choice == 3:
    s.peak()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Exiting program")
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
*IDLE Shell 3.14.0*
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/stack.py
Enter the size of the stack: 4

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 3
3 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
3 popped from the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

Error Table:

Error	Rectification
In peek, self.peek is taken in 'if' does not give correct answer for stack underflow.	Self.top is taken in 'if' to give correct output.

Description:

- The Stack class implements a stack using a Python list.
- size stores the maximum size of the stack.
- stack stores the stack elements.
- top keeps track of the top position. Initially, top = -1, meaning the stack is empty.
- **Push** adds an element to the top of the stack.
- **Pop** removes the top element from the stack.

- **Peek** displays the top element without removing it.
- **Display** prints all elements from top to bottom.
- **Stack Overflow** occurs when trying to push into a full stack.
- **Stack Underflow** occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b)

Aim:

➤ Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List.

Program:

```
# Stack using Linked List
# CSE25105
```

```
# Node Class
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
# Stack Class
```

```
class Stack:
    def __init__(self):
        self.top = None
```

```
# Push Operation
```

```
def push(self, item):
    new = Node(item)
    new.next = self.top
    self.top = new
```

```
    print(item, "pushed into the stack")
```

```
# Pop Operation
```

```
def pop(self):
    if self.top is None:
        print("Stack Underflow")
    else:
        temp = self.top
        self.top = self.top.next
```

```
    print(temp.data, "popped from the stack")
```

```
# Peek Operation
```

```
def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element:", self.top.data)
```

```
# Display Operation
```

```
def display(self):
    if self.top is None:
        print("Stack is empty")
    else:
```

```
        print("The elements of the stack:")
```

```
        temp = self.top

        while temp is not None:
            print(temp.data)
            temp = temp.next

# Main Program

s = Stack()

while True:

    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Exiting program")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 1
```

```
Enter the element to push: 4
```

```
4 pushed into the stack
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 2
```

```
4 popped from the stack
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 3
```

```
Stack is empty
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice: 4
```

```
Stack is empty
```

```
----- STACK MENU -----
```

1. Push
2. Pop
3. Peek
4. Display
5. Exit

```
Enter your choice:
```

Error Table:

Error	Rectification
No major error in the linked-list implementation.	Not required.

Description:

- The stack is implemented using a **singly linked list**.
- Each node contains data and a next pointer.
- top points to the top node of the stack.
- **Push** inserts a new node at the beginning of the linked list.
- **Pop** removes the node pointed to by top.
- **Peek** displays the data of the top node without removing it.
- **Display** traverses the linked list from top to the last node.
- If top is None, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.

Lab-7

1.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using array

Program:

```
# Queue Implementation using Linked List
# Operations: Enqueue, Dequeue, Peek, Display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow! Queue is empty.")
            return

        value = self.front.data
        self.front = self.front.next
```

```

        if self.front is None:
            self.rear = None

        print(value, "deleted from queue.")

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return

    current = self.front

    print("Queue elements:", end=" ")

    while current is not None:
        print(current.data, end=" ")
        current = current.next

    print()

# Create Queue object
queue = Queue()

# Menu-driven program
while True:
    print("\n----- QUEUE USING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to enqueue: "))
        queue.enqueue(value)

    elif choice == 2:
        queue.dequeue()

    elif choice == 3:
        queue.peek()

    elif choice == 4:
        queue.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice! Please try again.")

```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/queuelinkedlist.py =====
----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 24
24 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 45
45 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
24 deleted from queue.
```

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 2

24 deleted from queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 3

Front element: 34

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 4

Queue elements: 34 45

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 5

Program terminated.

Explanation:

A queue is a linear data structure that follows the FIFO (First In, First Out) principle. In this program, Python's list is used as an array to store queue elements.

Error table:

S.No	Error	Rectification
1	Using <code>queue.pop()</code> instead of <code>queue.pop(0)</code> removes the last element instead of the front element.	Use <code>queue.pop(0)</code> for dequeue operation.
2	Not checking the queue size before insertion can cause the queue to exceed its maximum size.	Check if <code>len(queue) == MAX_SIZE</code> before enqueue.
3	Accessing <code>queue[0]</code> when the queue is empty causes an <code>IndexError</code> .	Check if <code>len(queue) == 0</code> before performing Peek.

2.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using linked-list data structures.

Program:

```
# Queue Implementation using Linked List
# Operations: Enqueue, Dequeue, Peek, Display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow! Queue is empty.")
            return

        value = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        print(value, "deleted from queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty.")
        else:
            print("Front element:", self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty.")
            return

        current = self.front

        print("Queue elements:", end=" ")
```

```

        while current is not None:
            print(current.data, end=" ")
            current = current.next

        print()

# Create Queue
queue = Queue()

# Menu-driven program
while True:
    print("\n----- QUEUE USING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to enqueue: "))
        queue.enqueue(value)

    elif choice == 2:
        queue.dequeue()

    elif choice == 3:
        queue.peek()

    elif choice == 4:
        queue.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice! Please try again.")

```

OUTPUT:

===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/QUEUE.PY

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 78
78 inserted into queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 40
40 inserted into queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from queue.

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 78

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 78 40

----- QUEUE USING LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.

|

Explanation:

A linked-list queue consists of dynamically created nodes. Each node contains two parts:

Two pointers are maintained:

Front: Points to the first node.

Rear: Points to the last node.

Error table:

S.No	Error	Rectification
1	After deleting the last node, <code>rear</code> may still point to the deleted node.	If <code>front</code> becomes <code>None</code> , set <code>self.rear = None</code> .
2	Displaying the linked list without moving to the next node can create an infinite loop.	Use <code>current = current.next</code> inside the display loop.

3.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue Arrays structures.

Program:

```
# Circular Queue Implementation using Array
# Operations: Enqueue, Dequeue, Peek, Display

MAX_SIZE = 5

queue = [None] * MAX_SIZE
front = -1
rear = -1

while True:

    print("\n----- CIRCULAR QUEUE -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    # Enqueue
    if choice == 1:

        # Check whether queue is full
        if (rear + 1) % MAX_SIZE == front:
            print("Queue Overflow! Circular Queue is full.")

        else:
            value = int(input("Enter element to enqueue: "))

            # First element
            if front == -1:
                front = 0
                rear = 0

            else:
                rear = (rear + 1) % MAX_SIZE

            queue[rear] = value

            print(value, "inserted into circular queue.")

    # Dequeue
    elif choice == 2:

        if front == -1:
            print("Queue Underflow! Circular Queue is empty.")

        else:
            value = queue[front]

            print(value, "deleted from circular queue.")

            # Only one element was present
            if front == rear:
                front = -1
```

```
        rear = -1

    else:
        front = (front + 1) % MAX_SIZE

# Peek
elif choice == 3:

    if front == -1:
        print("Queue is empty.")

    else:
        print("Front element:", queue[front])

# Display
elif choice == 4:

    if front == -1:
        print("Queue is empty.")

    else:
        print("Circular Queue elements:", end=" ")

        i = front

        while True:
            print(queue[i], end=" ")

            if i == rear:
                break

            i = (i + 1) % MAX_SIZE

        print()

# Exit
elif choice == 5:
    print("Program terminated.")
    break

# Invalid choice
else:
    print("Invalid choice! Please try again.")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/CIRCULARQUEUE.PY ==
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 34
34 inserted into circular queue.
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 48
48 inserted into circular queue.
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 46
46 inserted into circular queue.
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
34 deleted from circular queue.
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 48
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 48 46
```

```
----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

A circular queue is a queue in which the last position is connected back to the first position.

Unlike a normal linear queue, a circular queue can reuse empty positions created after dequeue operations.

Error table:

S.No	Error	Rectification
1	Using <code>rear + 1</code> without <code>% MAX_SIZE</code> can make the rear go outside the array.	Use <code>rear = (rear + 1) % MAX_SIZE</code> .
2	Not resetting both pointers after deleting the last element can leave the queue in an incorrect state.	When <code>front == rear</code> , set both <code>front = -1</code> and <code>rear = -1</code> .

4.Aim:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue linked-list data structures.

Program:

```
# Circular Queue using Singly Linked List
# Operations: Enqueue, Dequeue, Peek, Display
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        # Queue is empty
        if self.rear is None:
            self.rear = new_node
            self.rear.next = self.rear

        else:
            # New node is inserted after rear
            new_node.next = self.rear.next
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into circular queue.")

    # Dequeue operation
    def dequeue(self):
        if self.rear is None:
            print("Queue Underflow! Queue is empty.")
            return

        # Front node
        front = self.rear.next

        # Only one node
        if self.rear == front:
            self.rear = None

        else:
            self.rear.next = front.next

        print(front.data, "deleted from circular queue.")

    # Peek operation
    def peek(self):
        if self.rear is None:
            print("Queue is empty.")

        else:
            print("Front element:", self.rear.next.data)

    # Display operation
```

```
def display(self):
    if self.rear is None:
        print("Queue is empty.")
        return

    front = self.rear.next
    current = front

    print("Circular Queue elements:", end=" ")

    while True:
        print(current.data, end=" ")
        current = current.next

        if current == front:
            break

    print()
```

```
# Create circular queue
queue = CircularQueue()
```

```
# Menu-driven program
while True:
```

```
    print("\n----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        value = int(input("Enter element to enqueue: "))
        queue.enqueue(value)
```

```
    elif choice == 2:
        queue.dequeue()
```

```
    elif choice == 3:
        queue.peek()
```

```
    elif choice == 4:
        queue.display()
```

```
    elif choice == 5:
        print("Program terminated.")
        break
```

```
    else:
        print("Invalid choice! Please try again.")
```

OUTPUT:

```
===== RESTART: C:/Users/Varshini Chillara/AppData/Local/Programs/Python/Python314/DSA/CIRCULARQUEUELINKED.PY ==
----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 36
36 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 86
86 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 57
Invalid choice! Please try again.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 67
67 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
```

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 2

36 deleted from circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 3

Front element: 86

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 4

Circular Queue elements: 86 67

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 5

Program terminated.

|

Explanation:

data → stores the element.

next → stores the address/reference of the next node.

This inserts the new node at the rear while maintaining the circular connection.

Error table:

S.No	Error	Rectification
1	Setting rear.next = None breaks the circular connection.	Set rear.next = front so the last node points to the first node.
2	Not setting rear = None when the only node is deleted leaves an invalid queue.	If rear == front, set rear = None.

